Dharmveer Bharti

swansi.dharmveer@gmail.com | 9572-234-299| linkedin.com/in/dharmveer-bharti-55b7231a7 | github.com/d-Bharti001

SUMMARY

Web3 backend engineer focused on **scalability** and **data integrity** (validation, idempotency, safe retries)
Built and owned reliable backend systems, APIs, and data pipelines in **Node.js**, **Golang**, and **Python**
Built Ethereum smart contracts in **Solidity**, and on-chain/off-chain **integration layers**
Skilled in **agentic AI systems**: multi-agent orchestration, tool calling, and LLM-integrated workflows

WORK EXPERIENCE

LCX | Software Development Engineer — Gurugram, IN | Aug 2022 - Mar 2026

- Led the backend development of Tiamonds (now TotoFinance), an NFT marketplace. Owned the backend end to end: scoping requirements, estimating timelines, smart contracts & API design, implementation, and deployment.
- Built **concurrency-aware REST APIs** and **asynchronous pipelines** in Node.js
- Refactored backend code and optimized database queries; **cut process memory consumption by 50%** (130 MB to 65 MB)
- Researched and implemented **server-signed message flows** for ERC-721 marketplace operations to reduce on-chain overhead (**cut gas costs by approximately 50%**) and to enable efficient workflows for new features
- Built an **automation pipeline (Python, Pandas)** that syncs approximately 80,000 CSV rows into MongoDB for over 20,000 assets
- Built a withdrawals indexer service for **OP Stack Layer 2 blockchains** with **failure-resilient retries** and **idempotent database writes**
- Built **retry-safe blockchain monitoring services** to synchronize and update the database based on blockchain events
- Designed NFT marketplace smart contracts supporting **server-signed messages**, and custom tokens, including the upgradeable LCX Token 2.0, live on Ethereum mainnet
- Collaborated with the frontend and design teams for feature integration and maintained backend deployments

Independent Software Engineer — Mar 2026 - Present

- **Distributed systems architecture:** implemented microservices isolation patterns, focusing on data consistency under concurrency and retries using fixed row-lock ordering, transactional outbox delivery, and gRPC communication
- Implemented **asynchronous event processing pipelines** utilizing Kafka and BullMQ/Redis
- **Agentic workflow orchestration**: built multi-agent pipelines using LangGraph, integrating LLM tool calling and LangSmith tracing

SKILLS

Languages: JavaScript, TypeScript, Go (Golang), Python, SQL, Solidity

Backend: Node.js, Express.js, REST APIs, gRPC, Kafka, BullMQ, Microservices

Datastores: MySQL, MongoDB, PostgreSQL, Redis

Web3 Tools & Frameworks: ethers.js, web3.js, Foundry, Hardhat, Truffle, Etherscan

Web3 Standards & Concepts: ERC-20, ERC-721, ERC-1155, EIP-712, Proxies and Upgrade Patterns

DevOps & Infra: Git, Linux, CI/CD (GitHub Actions), AWS (EC2, S3), Docker, Kubernetes, Nginx, systemd, PM2

LLM & AI Tools: Agentic AI, LLM orchestration, LangChain, LangGraph, LangSmith, Local AI (Ollama, Qwen), Claude Code

Web Scraping & Automation: Playwright, Crawl4AI

Fundamentals: Data Structures and Algorithms, System Design

PROJECTS

OP STACK WITHDRAWALS INDEXER — Golang, Optimism Stack, Blockchain, PostgreSQL

- Withdrawals indexer / tracker for OP Stack-based Layer 2 blockchains
- Two services (**Indexer** + **API**) with a shared PostgreSQL database, designed for horizontal scalability
- **Failure-safe indexing** with **retries/backoff**, **idempotent SQL writes** to prevent duplicates, and graceful shutdown
- Used Go interfaces to decouple core logic from chain and database implementations for cleaner code organization
- **Live UI**: https://op-sepolia-bridge.d-bharti.dev

PAYMENT MICROSERVICES APP — TypeScript, Node.js, MySQL, gRPC, Kafka, Kubernetes

- Payment system built as **containerized microservices** (API gateway, auth, money movement, ledger, email), deployed on a Kubernetes cluster
- Safe money transfers under concurrency and retries via **idempotency keys**, **atomic conditional balance updates**, **transactional database updates**, and **fixed row-lock ordering** to avoid deadlocks
- Asynchronous, fault-tolerant event delivery to downstream services using **Kafka** with the **transactional outbox pattern**
- Typed service-to-service communication using **gRPC** (Protobuf)
- Unit and integration tests (**Vitest**, **Testcontainers**) covering concurrent transfers, idempotent retries and outbox delivery
- **CI** with **GitHub Actions**: per-service test and build pipeline

CODEBASE-AWARE PR REVIEW AGENT TypeScript, Node.js, LangGraph, BullMQ, Redis, MySQL, Docker

- **Event-driven GitHub App** that reviews pull requests with a **LangGraph multi-agent pipeline** and posts one findings comment per PR, with severity levels and line permalinks
- **Reliable webhook processing**: signature verification, **idempotency** by GitHub delivery ID, **BullMQ** queue with retries, and stale-job detection when a PR changes during a run
- Specialized **security**, **quality** and **tests** agents with codebase tools (read file, search code), selected per diff; large diffs are chunked and agent loops have call and token limits
- **Swappable LLM provider** (Anthropic, OpenAI, Gemini, OpenRouter, Ollama) with **LangSmith** tracing and per-step token, latency, and model tracking in **MySQL**
- **CI/CD on GitHub Actions**: unit and Testcontainers integration tests, **approval-gated deployment** to a Linux server with health check and rollback, plus a Docker Compose setup for containerized deployments

EDUCATION

B. Tech. Computer Science and Engineering JH, IN | 2022
BIT Sindri | 9.24 CGPA

COMMUNITY CONTRIBUTIONS

- Smart Contract Security Audits
  - CodeHawks Profile: https://profiles.cyfrin.io/u/unitedcoolness26
  - 1 High finding, 2 Low findings - RAAC Audit
- Open-source contributions (**MeshJS**, TypeScript):
  - Refactored transaction metadata handling and fixed metadata-loss bug (PR 416)
  - Fixed HD wallet account bug (PR 396)
  - Reported wallet ownership verification issue (Issue 345)
- Technical articles on **Medium** (@dharmveerbharti): Off-chain Signature Workflow, Server-signed Messages in Web3